JOBSHEET 12

INCREMENTAL STATIC REGENERATION (ISR)

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_12

C. Implementasi ISR Otomatis

Bagian 1 – Tambahkan revalidate

- Buka halaman static.tsx pada folder src/pages/produk

```
import TampilanProduk from "../../views/produk";
import { ProductType } from "../../types/product.type";

const halamanProdukStatic = (props: { products: ProductType[] }) => {
  const { products } = props;
  return (
    <div>
        <h1>Halaman Produk Static</h1>
      <TampilanProduk products={products} />
    </div>
  );
};

export default halamanProdukStatic;

export async function getStaticProps() {
  const res = await fetch("http://127.0.0.1:3000/api/produk");
  // const response: { data: ProductType[] } = await res.json();
  const response: { data: ProductType[] } = await res.json();

  return {
    props: {
      products: response.data,
    },
    revalidate: 10, // Revalidate data setiap 10 detik
  };
}
```

Artinya:

- Setiap 10 detik halaman akan dicek ulang
- Jika ada perubahan data → cache diperbarui

Bagian 2 – Pengujian ISR

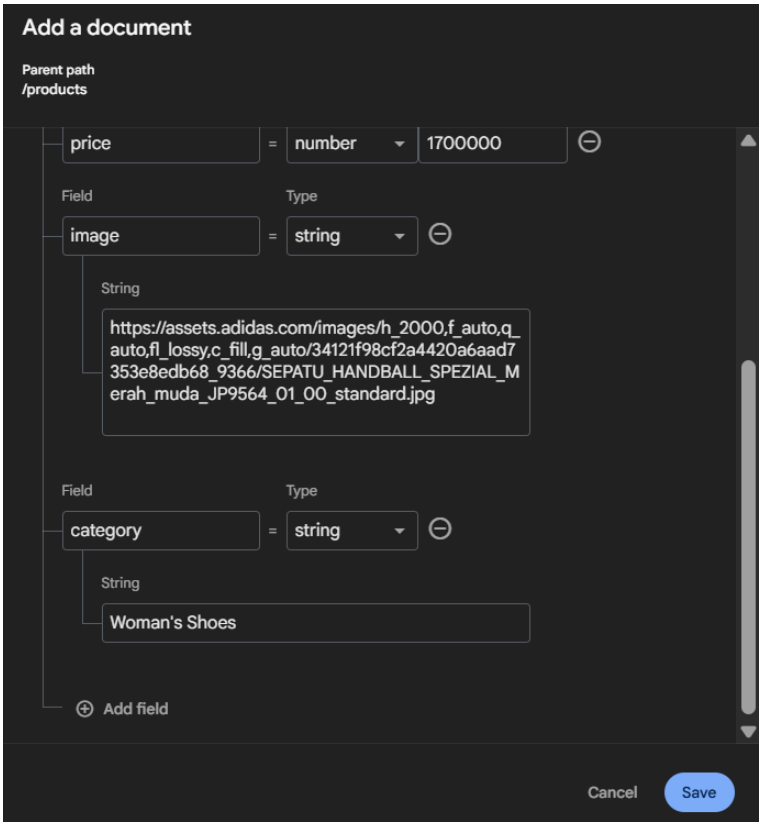
1. Jalankan: (lakukan hal sama seperti JS sebelumnya untuk ngebuild SSG)
    - npm run build (jika berhasil)

```
PS D:\Pemograman_Framework\praktikum_12\my-app-12> npm run build
│ ├ /produk/XimfFasRQKWqu6Bww2dV
│ ├ /produk/YoIN86GrgCwHYFpXL6vg
│ ├ /produk/fU1nrGvM6iHl79nGqrZq
│ └ /produk/hviDGTeFjhUU3IUpLa9g
├ ƒ /produk/server
├ ● /produk/static (384 ms)                    10s      1y
├ ○ /profile
├ ○ /profile/edit
├ ○ /setting/app
├ ○ /shop/[[...slug]]
├ ○ /user
└ ○ /user/password

○  (Static)   prerendered as static content
●  (SSG)      prerendered as static HTML (uses getStaticProps)
ƒ  (Dynamic)  server-rendered on demand
```

- npm run start

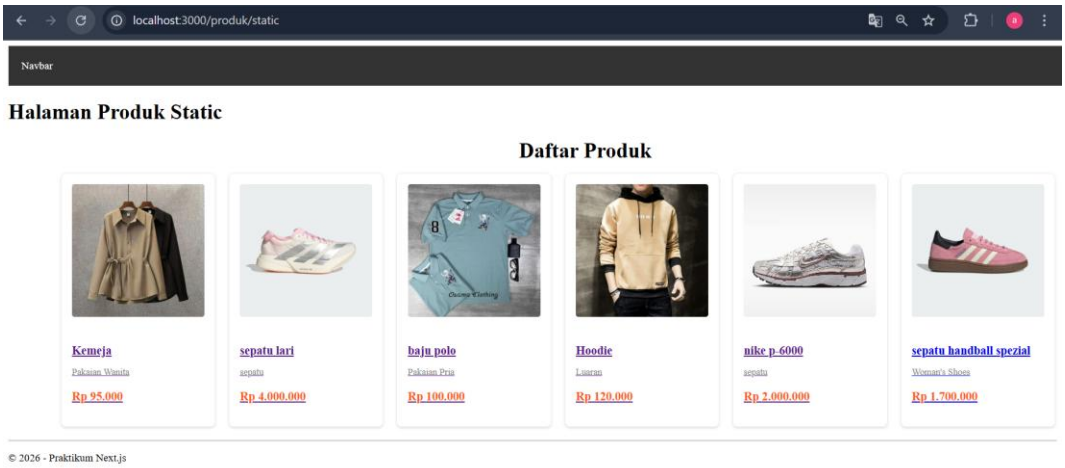
```
PS D:\Pemograman_Framework\praktikum_12\my-app-12> npm run start

  > my-app@0.1.0 start
  > next start

  ▲ Next.js 16.1.6
  - Local:        http://localhost:3000
  - Network:      http://192.168.56.1:3000

  ✓ Starting...
  ✓ Ready in 608ms
```

2. Tambahkan data baru di database pada firebase



3. Refresh halaman sebelum 10 detik → Data lama.
    - Sebelum 10 detik data yang akan ditampilkan masih data lama

4. Refresh setelah 10 detik → Data baru muncul.



D. On-Demand Revalidation

Jika tidak ingin menunggu waktu revalidate, gunakan endpoint khusus.

Bagian 1 – Buat API Revalidate

- Buat file revalidate.ts pada folder pages/api/ dan modifikasi

```typescript
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
  revalidated: boolean;
};

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>,
) {
  try {
    await res.revalidate("/produk/static");
    return res.status(200).json({ revalidated: true });
  } catch (error) {
    console.error("Error in API route:", error);
    res.status(500).send({ revalidated: false });
  }
}

```

Note : jika kita menggunakan code ini ada sedikit masalah yaitu Ketika salah satu data dihapus maka data yang terhapus tidak secara otomatis terupdate pada layar meskipun sudah direfresh selama 10 detik. Untuk bisa terupdate data yang ditampilkan diperlukan trigger dengan mengakses apinya http://localhost:3000/api/produk

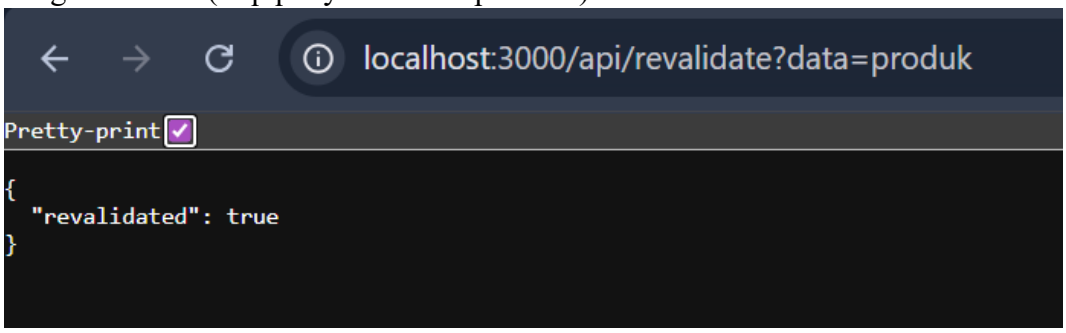
Bagian 2 – Tambahkan Parameter Data

Untuk mengatasi hal tersebut (pada bagian 1) maka suatu kondisi pada file revalidate.ts

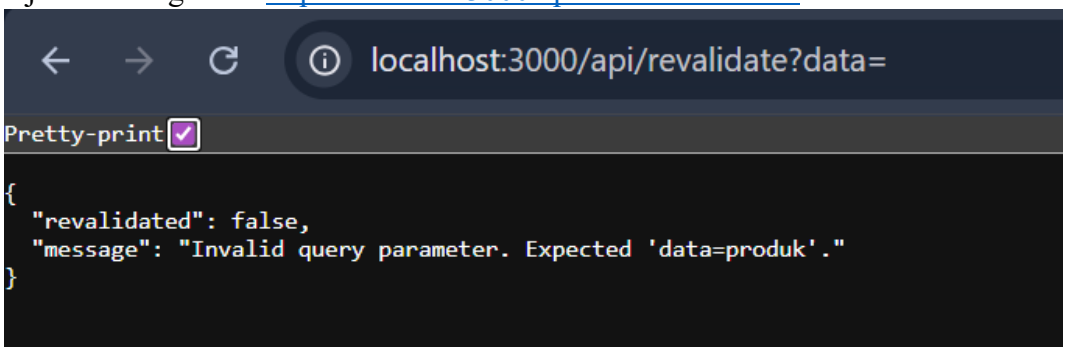
- Modifikasi file revalidate.ts

```typescript
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
  revalidated: boolean;
  message?: string;
};

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>,
) {
  if (req.query.data === "produk") {
    try {
      await res.revalidate("/produk/static");
      return res.status(200).json({ revalidated: true });
    } catch (error) {
      console.error("Error in API route:", error);
      res.status(500).send({ revalidated: false });
    }
  }

  return res.json({
    revalidated: false,
    message: "Invalid query parameter. Expected 'data=produk'.",
  });
}

```

- Uji coba menambahkan parameter dan value pada url http://localhost:3000/api/revalidate?data=produk maka akan muncul true dan sesuai dengan kondisi (req.query.data ===”produk”)



- Uji coba dengan url http://localhost:3000/api/revalidate?data=



Bagian 3 – Tambahkan Token Security

Perlu ditambahkan paramater agar user tidak merubah data melalui url

- Buka file .env dan modifikasi

```
8   REVALIDATE_TOKEN=12345678
9
```

- Modifikasi file revalidate.ts tambahkan kondisi pada line 13 – 17

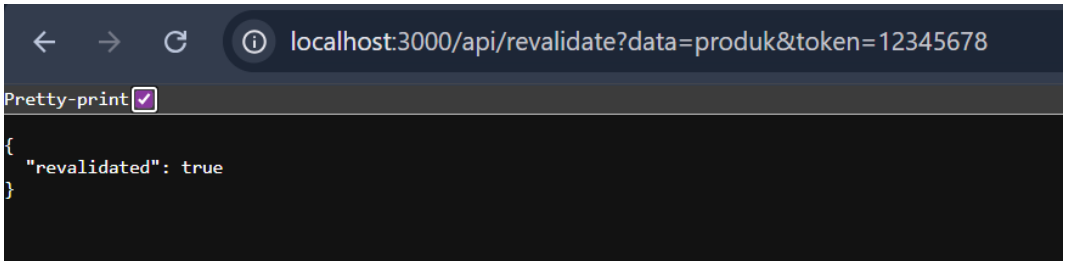
```typescript
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
  revalidated: boolean;
  message?: string;
};

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>,
) {
    if (req.query.token !== process.env.REVALIDATE_TOKEN) {
    return res.status(401).json({
        revalidated: false,
        message: "Insert correct token",
  });
}

  if (req.query.data === "produk") {
    try {
      await res.revalidate("/produk/static");
      return res.status(200).json({ revalidated: true });
    } catch (error) {
      console.error("Error in API route:", error);
      res.status(500).send({ revalidated: false });
    }
  }

  return res.json({
    revalidated: false,
    message: "Invalid query parameter. Expected 'data=produk'.",
  });
}

```

E. Pengujian Manual Revalidation

Akses:

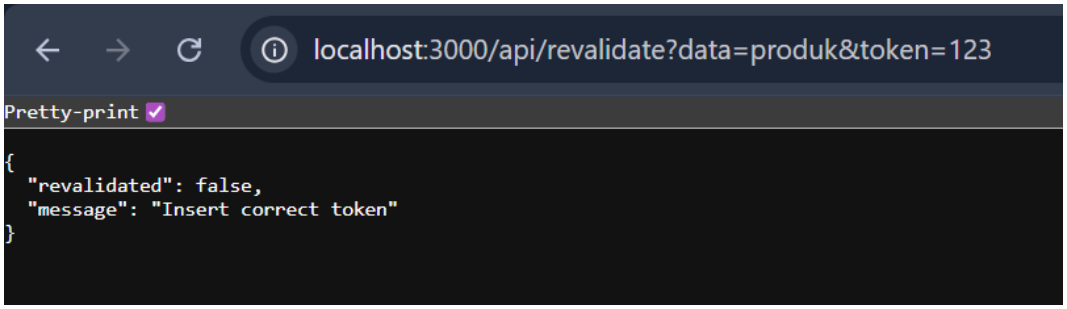
http://localhost:3000/api/revalidate?data=products&token=12345678

Jika benar:



{ revalidate: true }

Jika token salah:

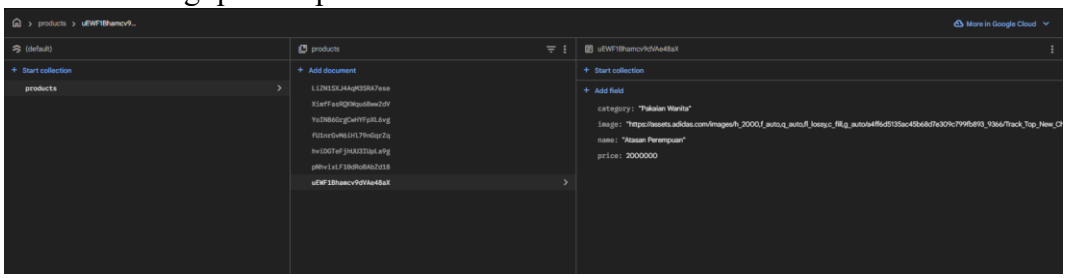


F. Perbandingan SSG vs ISR

Aspek	SSG	ISR
Update Data	Harus build ulang	Otomatis / Trigger
Cache	Static permanen	Static + Refresh
Cocok untuk	Konten tetap	Konten semi-dinamis

Tugas Individu

1. Tambahkan lagi produk pada firebase



2. Implementasikan ISR dengan revalidate: 10.
   pages/produk/static.tsx

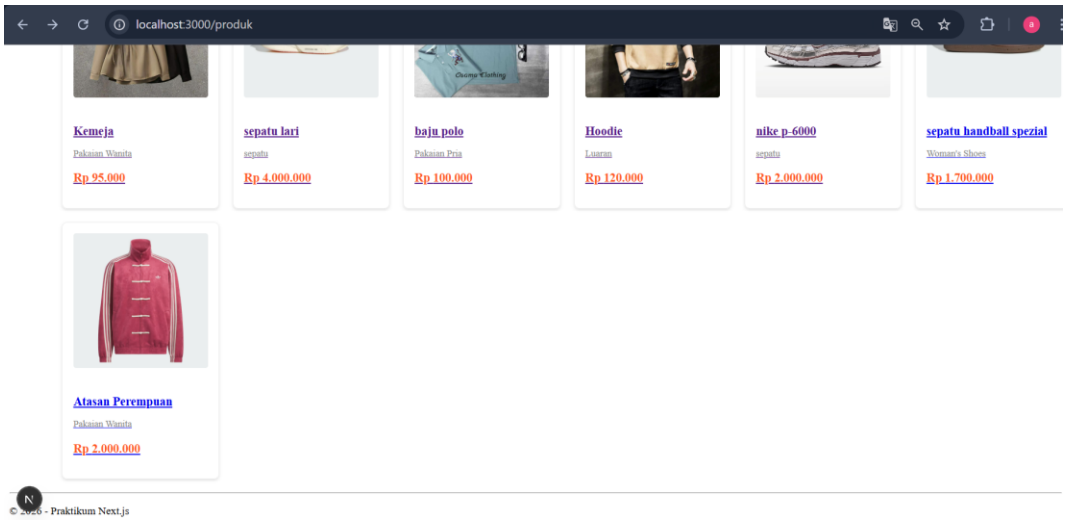
```python
import TampilanProduk from "../../views/produk";
import { ProductType } from "../../types/product.type";

const halamanProdukStatic = (props: { products: ProductType[] }) => {
  const { products } = props;
  return (
    <div>
        <h1>Halaman Produk Static</h1>
      <TampilanProduk products={products} />
    </div>
  );
};

export default halamanProdukStatic;

export async function getStaticProps() {
  const res = await fetch("http://127.0.0.1:3000/api/produk");
  // const response: { data: ProductType[] } = await res.json();
  const response: { data: ProductType[] } = await res.json();

  return {
    props: {
      products: response.data,
    },
    revalidate: 10, // Revalidate data setiap 10 detik
  };
}
```

3. Tambahkan endpoint On-Demand Revalidation.
pages/api/revalidate.ts

```python
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
  revalidated: boolean;
  message?: string;
};

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>,
) {
    if (req.query.token !== process.env.REVALIDATE_TOKEN) {
    return res.status(401).json({
        revalidated: false,
        message: "Insert correct token",
  });
}

  if (req.query.data === "produk") {
    try {
      await res.revalidate("/produk/static");
      return res.status(200).json({ revalidated: true });
    } catch (error) {
      console.error("Error in API route:", error);
      res.status(500).send({ revalidated: false });
    }
  }

  return res.json({
    revalidated: false,
    message: "Invalid query parameter. Expected 'data=produk'.",
  });
}
```

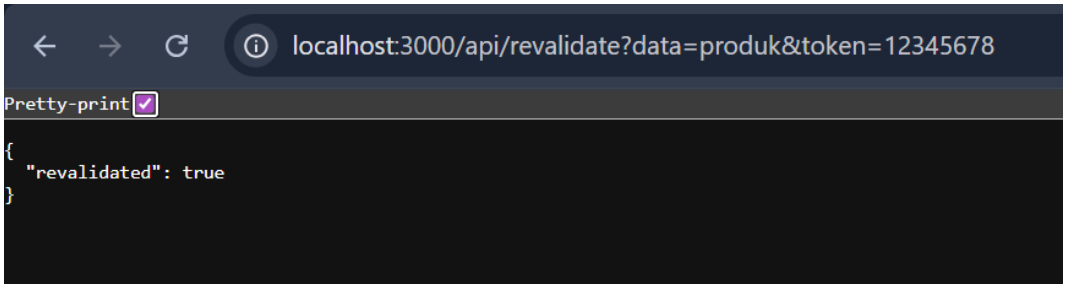
4. Tambahkan validasi token.

```
REVALIDATE_TOKEN=12345678
```

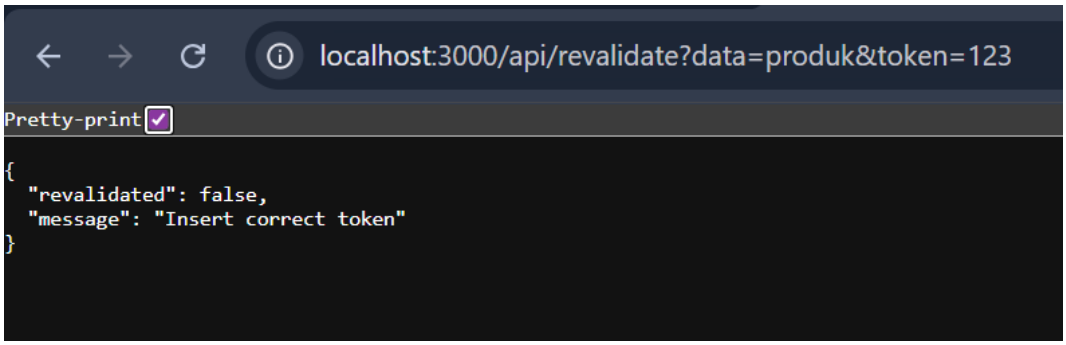


5. Uji dengan:

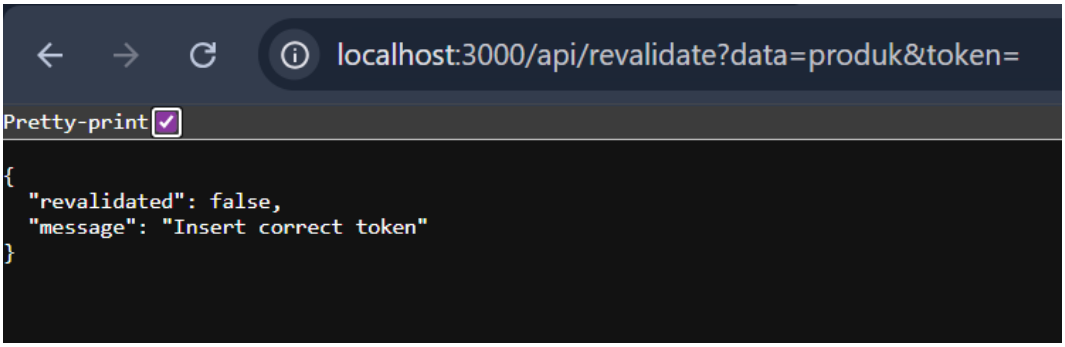
- Token benar



- Token salah



- Tanpa token



H. Pertanyaan Analisis

1. Mengapa ISR lebih fleksibel dibanding SSG?
   - ISR lebih fleksibel karena halaman yang sudah dibuat secara statis masih dapat diperbarui tanpa perlu melakukan build ulang seluruh aplikasi. Dengan adanya proses revalidate, data pada halaman dapat diperbarui secara berkala atau ketika ada permintaan tertentu. Hal ini membuat halaman tetap cepat seperti SSG tetapi tetap dapat menampilkan data yang lebih terbaru.

2. Apa perbedaan revalidate waktu dan on-demand?
   - Revalidate waktu dilakukan secara otomatis berdasarkan interval waktu yang telah ditentukan, misalnya setiap 10 detik. Sedangkan on-demand revalidation dilakukan secara manual melalui endpoint API ketika diperlukan, misalnya setelah terjadi perubahan data pada sistem. Dengan demikian, revalidate waktu bersifat otomatis sedangkan on-demand bersifat manual.

3. Mengapa endpoint revalidation harus diamankan?
   - Endpoint revalidation harus diamankan agar tidak dapat diakses oleh sembarang pengguna. Jika endpoint tersebut terbuka, maka siapa pun dapat memicu proses revalidation yang dapat mempengaruhi kinerja server. Oleh karena itu diperlukan mekanisme keamanan seperti penggunaan token agar hanya pihak tertentu yang dapat mengaksesnya.

4. Apa risiko jika token tidak digunakan?
   - Jika token tidak digunakan, maka endpoint revalidation dapat dipanggil oleh siapa saja. Hal ini berisiko menyebabkan penyalahgunaan, seperti melakukan revalidation secara terus-menerus yang dapat membebani server. Selain itu, pihak yang tidak berwenang juga dapat mengganggu proses pembaruan halaman pada aplikasi.

5. Kapan ISR lebih cocok dibanding SSR?
   - Jika token tidak digunakan, maka endpoint revalidation dapat dipanggil oleh siapa saja. Hal ini berisiko menyebabkan penyalahgunaan, seperti melakukan revalidation secara terus-menerus yang dapat membebani server. Selain itu, pihak yang tidak berwenang juga dapat mengganggu proses pembaruan halaman pada aplikasi.